BEHAVIORIQ: A REAL-TIME, LLM-AUGMENTED BEHAVIORAL INTELLIGENCE PLATFORM

A Minor Project Report submitted to

Manipal Academy of Higher Education

in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Computer Science (Data Science)

Submitted by

D V Phani Geethika(235816128), Namita Sampath(235816126), Dhanyatha Corry(235816448)

Under the guidance of
Aayushi Jain

SOCE

Manipal Institute of Technology

Bengaluru

<May/June/July> 2026



MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

CERTIFICATE

This is to certify that the minor project titled "BehaviorIQ: A Real-Time, LLM-Augmented Behavioral Intelligence Platform" is a record of the bonafide work done by <Student Name> (Reg. No. <Register No>) submitted in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology (B.Tech.) in INFORMATION TECHNOLOGY / CYBER SECURITY of Manipal Institute of Technology Bengaluru, Karnataka, (A Constituent Institute of Manipal Academy of Higher Education), during the academic year 2025-2026.

<Name of Internal Guide>	**Dr. <Present HOD Name>**
Designation, IT Dept. M.I.T, BENGALURU	HOD, IT Dept. M.I.T, BENGALURU

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to the Director and the Head of the Department of SOCE, Manipal Institute of Technology, Bengaluru, for providing the infrastructure and encouragement necessary to carry out this minor project.

I am deeply thankful to my project guide, Aayushi Jain, Assistant Professor(SOCE), Department of Information Technology, for the continuous guidance, valuable suggestions, and constant support extended throughout the duration of this project.

I also thank the laboratory in-charge and faculty members whose assistance was sought during the course of this work, as well as my peers and family for their encouragement and support.

ABSTRACT

Conventional clickstream analytics tools describe what happened in aggregate but cannot distinguish between two users who spend an identical amount of time on a product yet carry opposite intent. This project proposes BehaviorIQ, a real-time, LLM-augmented behavioral intelligence platform that ingests streaming user-interaction events, represents each browsing session as a learned behavioral "fingerprint" using a contrastively pretrained Transformer encoder, retrieves similar historical sessions through a production-grade vector database, predicts user intent using a retrieval-augmented classifier, personalizes the next-best-action using a contextual bandit, and allows stakeholders to query the entire system in plain English through a Retrieval-Augmented Generation (RAG) analytics copilot.

The system is designed around an industrial-style architecture consisting of a streaming ingestion layer (Kafka/Redpanda and Apache Flink), a feature store (Feast), an analytical warehouse (ClickHouse and PostgreSQL), a sequence-representation-learning module (a SASRec/BERT4Rec-style Transformer trained with a contrastive, InfoNCE-style self-supervised objective), a vector retrieval engine (Qdrant), a retrieval-augmented intent-prediction and contextual-bandit decisioning layer, an LLM-powered natural-language analytics copilot, and a full MLOps stack for experiment tracking, drift detection, explainability, and CI/CD.

The project is executed as thirteen dependency-ordered build stages, each with an explicit Definition of Done, and is evaluated against quantitative success criteria for every component, including streaming latency, feature-store parity, clustering quality, intent-prediction accuracy, retrieval precision, bandit reward, and copilot correctness. The work additionally proposes a publication-oriented research direction that isolates the effect of contrastive representation learning from that of retrieval augmentation on intent-prediction performance, and studies how early in a session each representation becomes reliable enough for real-time deployment.

TABLE OF CONTENTS

Contents	Page No.
Acknowledgements	I
Abstract	II
List of Tables	III
List of Figures	IV
Chapter 1: Introduction	1
Chapter 2: Literature Review	4
Chapter 3: Objectives	8
Chapter 4: Methodology	9
Chapter 5: Conclusion	17
References	19
Project Details	20

LIST OF TABLES

Table No.	Table Title	Page No.
4.1	Core Technology Stack	10
4.2	Differentiator Technology Layer	11
4.3	Canonical Event Vocabulary	12
4.4	Evaluation Framework and Success Bar for Every Component	16

LIST OF FIGURES

Figure No.	Figure Title	Page No.
4.1	BehaviorIQ End-to-End System Architecture	9

CHAPTER 1

INTRODUCTION

1.1 Introduction to the Area of Work

Behavioral analytics is the practice of collecting, modeling and interpreting how users interact with a digital product in order to understand and predict their intent. Most existing analytics tools are built around aggregate reporting: they answer questions such as how many users visited a page or what the average session duration was, but they do not model the order, timing and structure of user actions. Two users who spend an identical duration on a website can have entirely opposite intents; for example, one user may move directly from search to product to cart to purchase, while another may repeatedly compare products, view reviews and eventually abandon the session. Capturing this distinction requires treating a user session as an ordered sequence of events rather than as a bag of aggregate statistics.

This project, BehaviorIQ, is a real-time, LLM-augmented behavioral intelligence platform that is architected using the same patterns used by modern industrial data and machine-learning platform teams: streaming-first event ingestion, a feature store, transformer-based sequence representation learning, a production vector database, retrieval-augmented prediction, contextual-bandit personalization, and a Large Language Model (LLM) powered natural-language analytics copilot. The guiding idea of the system is that it should watch how a user moves through a product in real time, represent the session as a learned behavioral "fingerprint," retrieve and predict intent from that fingerprint, personalize decisions on the fly, and allow a human to simply ask the platform what is happening in plain English, without writing SQL or building dashboards.

1.2 Present-Day Scenario with Regard to the Work Area

Real-time streaming architectures built on Apache Kafka (or its lighter alternative Redpanda) together with Apache Flink form the backbone of virtually every serious modern data platform, spanning payments, advertising, fraud detection and recommendation systems. Transformer-based sequence models, following the SASRec and BERT4Rec family of architectures, are now the de-facto state of the art in industrial recommendation and intent-modeling systems, superseding earlier approaches based on hand-engineered session statistics. Vector databases and retrieval-augmented prediction, once a research curiosity, have become core infrastructure in nearly every modern personalization and semantic-search stack. Contextual bandits are the mechanism by which real platforms make personalization decisions under uncertainty, in contrast to static, rule-based targeting. Finally, LLM-powered analytics copilots built around Retrieval-Augmented Generation (RAG) are currently among the most in-demand capabilities in the industry, as they allow a conversational agent to answer business questions over a live data warehouse in a grounded, hallucination-checked manner.

A full MLOps stack, comprising experiment tracking, a model registry, drift monitoring, CI/CD and observability, is what separates a research notebook from an operable, monitorable and re-trainable production system. Every one of these individual components is now achievable by a small student team without specialized infrastructure, since the entire stack can be run locally through Docker Compose or on free-tier cloud credits.

1.3 The Problem

Conventional analytics tools answer what happened, in aggregate, but cannot distinguish between two users who spent an identical amount of time on a product yet had opposite intent. The core problem addressed by this project may be stated as follows: how can raw, streaming user-interaction events be transformed, in real time, into learned behavioral representations that support (a) similarity retrieval across millions of past sessions, (b) accurate intent prediction as a session unfolds, (c) automated personalization decisions, and (d) natural-language business querying, without hand-engineering features for every new question a stakeholder might ask.

Four capabilities follow directly from this problem statement, and each maps onto a subsystem of the proposed platform: understanding a session as it happens (real-time streaming ingestion and a feature store), comparing a session against history (a transformer session encoder together with a vector database), predicting what happens next (a retrieval-augmented intent model together with a contextual bandit), and letting a human simply ask a question (an LLM analytics copilot combining RAG with a text-to-SQL agent).

1.4 Brief Project Work Schedule

The project is executed as one continuous, dependency-ordered build spanning thirteen stages (Stage 0 to Stage 12), described in detail in Chapter 4. Work proceeds from environment setup and data instrumentation, through streaming ingestion, feature engineering, behavioral mining, sequence representation learning, vector retrieval, intent prediction and bandit-based personalization, to the LLM analytics copilot, MLOps instrumentation, dashboarding, and finally containerized deployment with CI/CD. Each stage is completed and verified against an explicit Definition of Done before the next stage begins, ensuring that the system is always in a working, testable state.

1.5 Organization of the Project Report

This report is organized into five chapters. Chapter 1 introduces the problem area, the present-day industrial context, the problem statement and the project schedule. Chapter 2 presents a review of the relevant literature and background theory underlying each major subsystem. Chapter 3 states the specific objectives of the project. Chapter 4 describes the complete system architecture, technology stack, data design and the thirteen-stage execution methodology, along with the evaluation framework used to assess every component. Chapter 5 concludes the report with a summary of the work, the methodology adopted, and the overall conclusions drawn, followed by the references used throughout the project.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction to the Project Title

BehaviorIQ combines six distinct research and engineering threads that are individually well established but not commonly integrated into a single, coherent student project: streaming data infrastructure, sequential/session-based recommendation, self-supervised contrastive representation learning, vector retrieval and retrieval-augmented prediction, contextual-bandit decision making, and LLM-based retrieval-augmented natural-language querying. This chapter reviews the present state of each area, the background theory that motivates the design choices made in Chapter 4, and summarizes the outcome of the review.

2.2 Present State / Recent Developments in the Work Area

Streaming clickstream analytics: Distributed log-based messaging systems such as Apache Kafka, together with stateful stream processors such as Apache Flink, have become the standard backbone for durable, ordered, high-throughput event pipelines in industry, replacing older micro-batch architectures for use cases that require sub-second freshness, such as fraud detection and real-time personalization.

Session-based and sequential recommendation: Self-attentive sequential recommendation (SASRec) and bidirectional session modeling with masked-item prediction (BERT4Rec) established that treating a user's interaction history as an ordered sequence, analogous to a sentence, and applying Transformer self-attention over it, substantially outperforms earlier Markov-chain and RNN-based approaches for predicting the next action or item of interest.

Self-supervised contrastive representation learning: Contrastive frameworks such as SimCLR for images and CLIP for image-text pairs popularized the InfoNCE objective, in which two augmented views of the same underlying instance are pulled together in embedding space while different instances are pushed apart. This same family of objectives has since been adapted to sequential and session data, allowing an encoder to learn useful behavioral representations from large volumes of unlabeled interaction data before any labeled outcome (such as purchase or abandonment) is available.

Vector databases and retrieval-augmented methods: Approximate nearest-neighbor vector search engines, such as Qdrant, FAISS-based systems and Milvus, are now core infrastructure for semantic search and Retrieval-Augmented Generation (RAG). The RAG paradigm, originally developed to ground language-model generation in retrieved documents, generalizes naturally to prediction tasks, where the outcomes of the most similar historical instances are fed as additional evidence into a downstream classifier.

Contextual bandits for personalization: Algorithms such as Thompson Sampling and LinUCB frame personalization as sequential decision-making under uncertainty, in which an action is chosen based on context, a reward is observed, and the decision policy is updated online. This decisioning framing is what differentiates a production personalization system from a static, offline-trained classifier.

LLM-powered analytics copilots: The combination of large language models with RAG and text-to-SQL agents has emerged as a widely adopted pattern for allowing non-technical stakeholders to query structured data warehouses using natural language, provided that appropriate guardrails (schema whitelisting, k-anonymity floors) are used to prevent hallucinated or privacy-violating answers.

2.3 Brief Background Theory

A user session is modeled as an ordered list of discrete events sharing a session identifier, closed after a period of inactivity or an explicit terminal event, whichever occurs first. Each event in the sequence is treated as a token drawn from a finite vocabulary (for example, product_view, add_to_cart, compare), so that an entire session can be tokenized in the same way a sentence is tokenized in natural language processing. A compact Transformer encoder then maps this tokenized sequence to a fixed-size embedding vector, and is pretrained using a contrastive, InfoNCE-style loss over augmented pairs of the same session (formed by masking, cropping or locally reordering events), so that semantically similar behavioral patterns land close together in embedding space even before any labeled outcome is used.

Once session embeddings exist, they are indexed in a vector database that supports approximate nearest-neighbor search with metadata filtering, allowing the system to retrieve, for any live or historical session, the top-K most similar past sessions along with their known outcomes. These retrieved outcomes are concatenated with the session's own embedding as additional input features to a downstream intent classifier, an approach that borrows the retrieval-augmentation idea from language generation and applies it to prediction. A contextual bandit then consumes the resulting intent estimate as context and selects a real-time personalization action, updating its policy as rewards (such as an eventual purchase) are observed. Finally, an LLM agent is given restricted, schema-aware tool access to the warehouse (via text-to-SQL) and the vector database (via semantic search), and is required to answer natural-language business questions by executing these tools and grounding its answer in the returned data rather than in memorized or hallucinated figures.

2.4 Literature Survey

The following works and system families were surveyed and directly motivate the architecture adopted in this project.

- Kang and McAuley's SASRec, and Sun et al.'s BERT4Rec, established self-attention as the state-of-the-art mechanism for session-based and sequential recommendation, replacing earlier Markov-chain and recurrent-network approaches.
- Chen et al.'s SimCLR and Radford et al.'s CLIP popularized the contrastive InfoNCE training objective, which this project adapts from images and image-text pairs to behavioral event sequences.
- Lewis et al.'s original Retrieval-Augmented Generation (RAG) paper for open-domain question answering motivates both the LLM analytics copilot (Stage 9) and the retrieval-augmented intent classifier (Stage 8), which applies the same retrieve-then-condition principle to a discriminative task instead of a generative one.
- Agrawal and Goyal's analysis of Thompson Sampling, and Li et al.'s LinUCB algorithm for contextual-bandit personalization, underlie the online decisioning layer used to choose real-time personalization actions.
- Srivastava et al.'s work on sequential pattern mining (PrefixSpan-style algorithms), Agrawal and Srikant's FP-Growth-style association-rule mining, and Campello et al.'s HDBSCAN density-based clustering algorithm underlie the exploratory behavioral-mining stage of the pipeline.
- Industrial engineering references on Apache Kafka, Apache Flink, Apache Spark and the Feast open-source feature store describe the streaming, batch and feature-serving patterns that this project's ingestion and feature layers are built on.

2.5 Summarized Outcome of the Literature Review

The literature review confirms that each individual component proposed for BehaviorIQ, streaming ingestion, Transformer-based sequence modeling, contrastive self-supervised pretraining, vector-based retrieval augmentation, contextual-bandit decisioning, and LLM-based RAG copilots, is independently well-established and actively used in industrial systems. However, the review also confirms that most existing student and academic projects in this space either stop at aggregate dashboarding, or use only one or two of these techniques in isolation. The specific combination proposed here, and in particular the retrieval-augmented intent classifier that fuses learned Transformer embeddings with nearest-neighbor outcome evidence, and the application of an LLM RAG copilot on top of a fully instrumented behavioral warehouse, represents an integration that is not commonly demonstrated at the student-project level and forms the basis of the research direction described in Chapter 4.

CHAPTER 3

OBJECTIVES

Based on the problem statement and the literature reviewed in Chapter 2, the specific objectives of this project are as follows.

1. To design and implement a real-time, streaming clickstream ingestion pipeline (using a demo storefront, a hand-written tracking SDK, Apache Kafka/Redpanda and Apache Flink) that captures user-interaction events with zero event loss and correct per-session ordering under sustained load.
2. To design a feature store (using Feast, backed by Redis for online serving and Parquet for offline storage) that guarantees training/serving parity between the exact same feature values used for real-time scoring and for offline model training.
3. To build an analytical warehouse (ClickHouse for fact tables and PostgreSQL for dimension tables) capable of answering canonical funnel, conversion and product-performance queries with verifiable correctness.
4. To perform behavioral data mining on the collected sessions using sequential pattern mining (PrefixSpan), association-rule mining (FP-Growth) and density-based clustering (HDBSCAN) in order to discover interpretable behavioral personas and patterns.
5. To design and train a compact, self-supervised, contrastively pretrained Transformer encoder that represents each user session as a fixed-size behavioral embedding, and to compare it against a generic sentence-embedding baseline.
6. To build a production-grade vector retrieval engine (Qdrant) that indexes every session embedding and supports fast (sub-100 ms), metadata-filtered nearest-neighbor search.
7. To build and compare four intent-prediction variants, aggregate-feature-based, hand-crafted-sequence-feature-based, learned-embedding-based, and a novel retrieval-augmented variant that fuses learned embeddings with the outcomes of retrieved nearest-neighbor sessions, and to quantify the improvement achieved by retrieval augmentation.
8. To implement a contextual-bandit personalization layer (Thompson Sampling) that selects a real-time next-best-action from the current intent estimate and improves its policy over time relative to a static, rule-based baseline.
9. To build an LLM-powered natural-language analytics copilot that combines a guarded text-to-SQL tool with a semantic vector-search tool, so that a business user can ask plain-English questions and receive grounded, non-hallucinated, data-backed answers.

10. To instrument the entire platform with a full MLOps stack, MLflow for experiment tracking and model registry, Prometheus and Grafana for observability, Evidently AI for drift detection, and SHAP for model explainability, so that the system is operable and monitorable, not merely demonstrable once.
11. To containerize and package the complete platform using Docker Compose with an automated CI/CD pipeline (GitHub Actions), so that the entire system can be reproduced from a clean machine with a documented, one-line command.

CHAPTER 4

METHODOLOGY

4.1 System Overview and Block Diagram

The proposed system is organized as a single, layered pipeline in which events flow from a demo storefront, through a streaming backbone, into a feature store and warehouse, and are then progressively enriched by behavioral mining, sequence representation learning, vector retrieval, retrieval-augmented intent prediction, contextual-bandit personalization, and an LLM analytics copilot, with an MLOps and observability layer cutting across every stage. The full block diagram is shown in Figure 4.1.

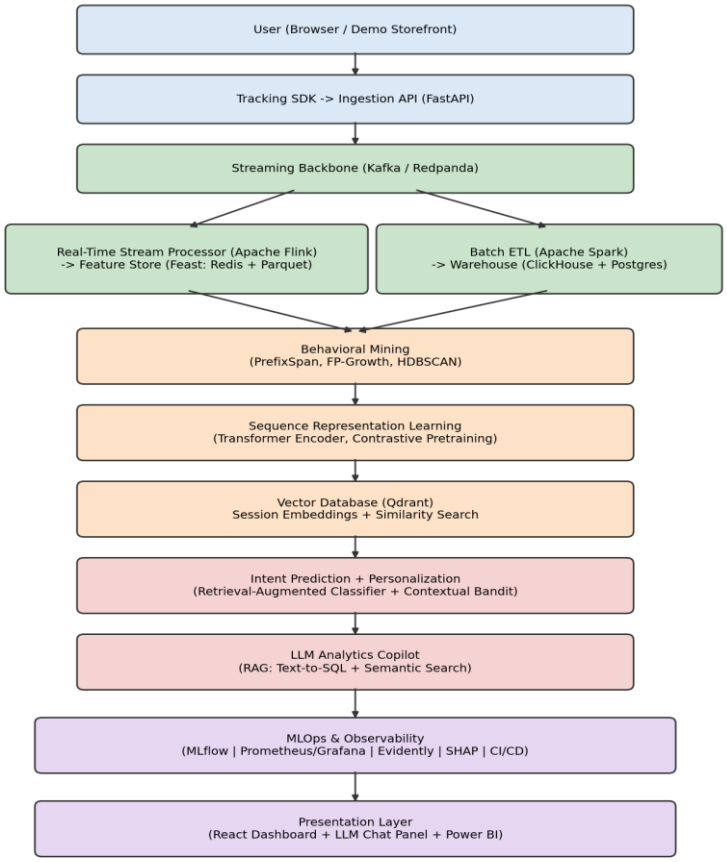


Figure 4.1: BehaviorIQ End-to-End System Architecture

Every arrow in Figure 4.1 corresponds to exactly one numbered build stage described in Section 4.4, built and verified in isolation before being wired to the next, so that no more than one architectural link is under construction at any time.

4.2 Detailed Methodology and Tools Used

The methodology followed is a staged, dependency-ordered build rather than a calendar-based schedule: each stage is completed only once its predecessor's Definition of Done has been verified. The complete technology stack is divided into a Core layer, without which the system does not function, and a Differentiator layer, which is added once the Core layer is verified stable, and is what distinguishes this project from a conventional aggregate-dashboard analytics system.

Table 4.1: Core Technology Stack

Layer	Technology	Rationale
Demo storefront	Next.js (React) + TypeScript + Tailwind CSS	Component-driven and fast to build; keeps development effort on the platform rather than on styling.
Tracking SDK	Hand-written TypeScript module	Gives full control over the event schema and client-side batching logic.
Ingestion API	Python, FastAPI, Pydantic	Asynchronous, strongly typed and auto-documented; an industry-standard choice for high-throughput APIs.
Streaming backbone	Apache Kafka (or Redpanda for local development)	The de-facto standard for durable, ordered, high-throughput event streams.
Real-time processing	Apache Flink	True low-latency, stateful stream processing with native session windowing.
Batch ETL	Apache Spark	Scales cleaning, deduplication and heavier offline feature computation.
Feature store	Feast (open source)	Guarantees identical feature values online and offline, eliminating training/serving skew.
Online feature cache	Redis	Sub-millisecond feature lookups for real-time scoring.
Analytical warehouse	ClickHouse	Columnar, real-time OLAP engine suited to high-volume event analytics.
Relational store	PostgreSQL	Home for dimension and configuration data.
Data mining	mlxtend, prefixspan, scikit-learn, HDBSCAN	Sequential pattern mining, association-rule mining and density-based clustering.

Table 4.2: Differentiator Technology Layer

Layer	Technology	Rationale
Sequence representation learning	Custom Transformer encoder (SASRec/BERT4Rec-style), PyTorch	State-of-the-art approach to modeling ordered behavior.
Self-supervised pretraining	Contrastive (InfoNCE-style) objective over augmented session pairs	Lets the encoder learn from unlabeled traffic before labeled outcomes exist.
Vector database	Qdrant	Self-hostable, production-grade vector search with metadata filtering.

Layer	Technology	Rationale
Retrieval-augmented prediction	Top-K nearest-neighbour outcomes fed into the intent model	Applies the RAG idea to prediction rather than text generation.
Personalization decisioning	Contextual bandit (Thompson Sampling / LinUCB)	Chooses an action under uncertainty and keeps learning from outcomes.
LLM analytics copilot	Claude/GPT API + RAG (text-to-SQL over ClickHouse + semantic search over Qdrant)	Answers business questions in natural language, grounded in live data.
Explainability	SHAP	Feature-level explanation for every intent prediction.
Experiment tracking / model registry	MLflow	Tracks every run and promotes the best model version to production.
Drift detection	Evidently AI	Detects when incoming traffic statistically diverges from training data.
Observability	Prometheus + Grafana	Live dashboards for pipeline health, latency, throughput and model performance.
CI/CD	GitHub Actions	Automated tests, linting and build on every push.
Containerization / orchestration	Docker Compose (local), Kubernetes (stretch)	Reproducible environment for every teammate and for the final demo.

4.3 Data Design, Event Schema and Assumptions

The demo storefront instruments the flow Homepage -> Search -> Product Listing -> Product Page -> Reviews -> Compare -> Cart -> Checkout -> Purchase. Every event emitted by the tracking SDK follows a canonical JSON schema containing an event identifier, a session identifier, an anonymized user identifier, an event type, a timestamp, the page and product context, the device, the referrer, and a metadata object (for example scroll depth and dwell time in milliseconds).

Table 4.3: Canonical Event Vocabulary

Event Types	Category
page_view, search, click, scroll, hover	Passive/navigational interaction events
product_view, review_view, compare	Product-consideration events
add_to_cart, remove_from_cart, checkout_start, purchase	Transaction-funnel events
exit	Session-termination event

A session is defined, once and for all, as the ordered list of events sharing a session identifier, closed after thirty minutes of inactivity or an explicit purchase/exit event, whichever occurs first; this definition feeds every downstream component, and changing it later would invalidate all prior embeddings and trained models. It is assumed that traffic used for development and evaluation is primarily synthetic, generated by a configurable traffic-simulation script with convert/abandon/browse probabilities, supplemented where possible with a smaller volume of real collected sessions.

Privacy is engineered in from the outset rather than retrofitted: no names, emails, passwords or free-text fields that could contain personally identifiable information are captured; all identifiers are randomly generated and never derived from real identity; k-anonymity-style aggregation is applied before any figure leaves the warehouse for the dashboard or the LLM copilot, so that no metric is ever surfaced from fewer than a defined minimum number of sessions; and, as an optional stretch goal, differential-privacy noise may be added to aggregate query results returned by the copilot.

4.4 Execution Plan: Thirteen Build Stages

The system is built as thirteen dependency-ordered stages, Stage 0 through Stage 12, each with an explicit Definition of Done.

12. Stage 0 - Environment, Repository and Contracts: establish the monorepo, the shared event-schema and session-construction contracts, a Docker Compose skeleton for all backing services, and a CI skeleton that lints and tests every pull request. Definition of Done: any teammate can clone the repository and bring up every backing service with one command.
13. Stage 1 - Demo Storefront and Tracking SDK: build the Next.js storefront, the TypeScript tracking SDK, and a synthetic traffic generator that simulates realistic sessions. Definition of Done: ten minutes of generated traffic produces thousands of correctly shaped events with realistic behavioral variety.
14. Stage 2 - Real-Time Ingestion and Streaming Backbone: a FastAPI endpoint validates and publishes events to a Kafka topic partitioned by session identifier, with a dead-letter topic for malformed events. Definition of Done: zero event loss and correct per-session ordering under sustained burst load.
15. Stage 3 - Real-Time Processing and the Feature Store: a Flink job maintains rolling per-session state and writes features to Feast (Redis online, Parquet offline); a Spark batch job backfills heavier aggregate features. Definition of Done: identical feature definitions serve both live scoring and historical training with no code duplication.
16. Stage 4 - Warehouse and OLAP Layer: ClickHouse fact tables and PostgreSQL dimension tables are populated by a continuous ETL job. Definition of Done: warehouse queries over a full day of traffic are verified by hand-spot-checking against raw event logs.
17. Stage 5 - Behavioral Data Mining: PrefixSpan for sequential patterns, FP-Growth for association rules, and HDBSCAN for density-based clustering are run over the collected sessions to discover interpretable behavioral personas. Definition of Done: a documented report of the top sequential patterns, association rules and labeled clusters, each backed by a concrete metric.

18. Stage 6 - Sequence Representation Learning: each session is tokenized as a sequence of events; a compact Transformer encoder is pretrained with a contrastive, InfoNCE-style objective over augmented session pairs, and a classification head is later fine-tuned on top for the labeled intent task, alongside a simpler sentence-embedding baseline. Definition of Done: nearest-neighbour sanity checks show behaviorally similar sessions clustering together in both embedding spaces, with the custom encoder visibly tighter.
19. Stage 7 - Vector Retrieval Engine: Qdrant indexes every session embedding with its outcome metadata and exposes a top-K nearest-neighbour retrieval API with metadata filtering. Definition of Done: queries return predominantly outcome-consistent neighbours in well under 100 milliseconds.
20. Stage 8 - Retrieval-Augmented Intent Prediction and Contextual-Bandit Personalization: four intent-prediction variants (aggregate features, hand-crafted sequence features, learned embeddings, and a retrieval-augmented variant that fuses embeddings with retrieved-neighbour outcomes) are trained and compared, and a Thompson Sampling contextual bandit selects and learns from real-time personalization actions. Definition of Done: a comparison table across all four variants, and a bandit that visibly shifts its action distribution as it accumulates reward.
21. Stage 9 - LLM-Powered Natural-Language Analytics Copilot: a text-to-SQL tool (restricted to whitelisted views) and a semantic-search tool over Qdrant are wired into an agent loop that routes each natural-language question to the correct tool, enforces the k-anonymity floor, and produces a grounded answer. Definition of Done: at least fifteen diverse pre-written test questions are answered correctly with zero hallucinated numbers.
22. Stage 10 - MLOps, Monitoring and Explainability: MLflow logs and registers every model version; Prometheus and Grafana instrument the ingestion, streaming and prediction services; Evidently AI compares live and training feature distributions; SHAP explains every intent prediction. Definition of Done: live dashboards, an injected-drift detection report, and SHAP explanations for arbitrary predictions all function during a demo run.
23. Stage 11 - Dashboard and Demo Experience: a React dashboard integrates the funnel view, behavioral-segment map, intent-score feed, semantic search box and the embedded LLM chat panel, with an optional Power BI stakeholder view. Definition of Done: a scripted five-minute demo runs end-to-end on a freshly booted environment without manual intervention.
24. Stage 12 - Deployment and CI/CD: every service is containerized behind a single docker-compose.yml, with GitHub Actions running lint, unit tests and a smoke test on every push, and an optional Kubernetes/Terraform stretch goal. Definition of Done: a

clean machine running only Docker can bring up the entire platform from the repository using documented one-line commands.

4.5 Evaluation Framework and Success Bar

Each component of the system is evaluated against an explicit, quantitative success bar rather than being assessed subjectively, so that the overall platform's readiness can be verified objectively at every stage.

Table 4.4: Evaluation Framework and Success Bar for Every Component

Component	Metric(s)	Target
Streaming pipeline	Event loss rate; end-to-end latency	0% loss under burst load; < 2 s p95 event-to-warehouse latency
Feature store	Online/offline feature parity	100% match on spot-checked features
Sequential / association mining	Support, confidence, lift	Document top patterns with lift > 2.0
Clustering	Silhouette score, Davies-Bouldin index	Silhouette > 0.3 with human-interpretable clusters
Intent prediction (A/B/C/C+R)	Accuracy, Precision, Recall, F1, ROC-AUC, PR-AUC	C+R measurably beats A and B; report the exact delta
Semantic retrieval	Precision@K, Recall@K, MRR, NDCG	Precision@10 > 0.7 on held-out sessions
Contextual bandit	Cumulative reward vs. static-rule baseline	Bandit beats a fixed-rule baseline after a defined warm-up window
LLM copilot	Answer correctness on a fixed test-question set; hallucination rate	≥ 90% correct; zero unfounded numeric claims
Drift monitoring	Detected vs. injected drift events	100% detection on synthetic drift injections

4.6 Proposed Research Direction

Beyond the engineering deliverable, the project proposes a research study titled "Retrieval-Augmented Behavioral Intent Prediction: Combining Contrastively Pretrained Session Transformers with Nearest-Neighbour Evidence for Real-Time E-Commerce Decisioning." The study asks whether a contrastively pretrained Transformer session encoder separates purchase/abandon/browse outcomes better than hand-crafted aggregate features and a generic sentence-embedding baseline; whether augmenting a session's own embedding with outcome evidence from its top-K nearest historical neighbours measurably improves intent-prediction accuracy and calibration over embeddings alone; how early in a session (after 3, 5, 10 or 20 events) each representation becomes reliable enough for real-time use; and whether a contextual-bandit personalization layer accumulates measurably more reward than a static, rule-based baseline. The experimental design isolates representation quality from retrieval augmentation as two independently testable variables, which most comparable undergraduate work conflates into a single end-to-end comparison, and extends the evaluation into an actual downstream decisioning layer rather than stopping at predictive accuracy alone.

CHAPTER 5

CONCLUSION

5.1 Brief Summary of the Work

This project set out to design and build BehaviorIQ, a real-time, LLM-augmented behavioral intelligence platform that goes beyond conventional aggregate clickstream dashboards by modeling each user session as a learned, contrastively pretrained behavioral embedding, retrieving similar historical sessions through a production vector database, predicting intent using a retrieval-augmented classifier, personalizing decisions through a contextual bandit, and allowing natural-language querying of the entire system through an LLM-powered RAG analytics copilot, all underpinned by a full MLOps and observability stack.

5.2 Problem Statement / Objective, in Brief

The central problem addressed was that conventional analytics tools cannot distinguish between sessions of identical duration but opposite user intent, since they operate on aggregate statistics rather than on the ordered structure of behavior. The objective was therefore to transform raw, streaming interaction events, in real time, into learned representations that support similarity retrieval, intent prediction, automated personalization, and plain-English business querying, without hand-engineering features for every new analytical question.

5.3 Work Methodology Planned/Adopted, in Brief

The methodology adopted a thirteen-stage, dependency-ordered build, beginning with environment setup and instrumented data collection, proceeding through streaming ingestion, feature-store construction, warehouse design, behavioral mining, contrastive sequence-representation learning, vector retrieval, retrieval-augmented intent prediction, contextual-bandit personalization and an LLM analytics copilot, and concluding with MLOps instrumentation, a unified dashboard, and containerized, CI/CD-driven deployment. Every stage was tied to an explicit, verifiable Definition of Done and to a quantitative evaluation target, ensuring that progress could be objectively measured component by component rather than assessed only at the end.

5.4 Conclusion

The proposed architecture demonstrates that a small student team can build, in a single continuous effort, a system that mirrors the core architectural patterns of modern industrial data and machine-learning platforms, streaming-first ingestion, a feature store, transformer-based sequence modeling, production vector retrieval, retrieval-augmented prediction, contextual-bandit decisioning and an LLM analytics copilot, without requiring infrastructure beyond a laptop and free-tier cloud credits. The explicit separation of representation learning from retrieval augmentation, and the extension of evaluation into an actual downstream decisioning

layer, positions the project's headline contribution, the retrieval-augmented intent classifier, as a genuinely defensible and current research direction rather than a purely engineering exercise.

5.5 General Conclusion and Future Work

Future extensions of this work could include deploying the stretch-goal Kubernetes and Terraform infrastructure-as-code layer for full cloud-native operation, extending the differential-privacy guarantee on the LLM copilot's aggregate answers, incorporating additional modalities (such as product images or textual reviews) into the session encoder, and running the full research evaluation described in Section 4.6 on real, non-synthetic user traffic once available, to validate whether the gains observed on simulated data generalize to production conditions.

REFERENCES

Journal / Conference Papers

[1] W. Kang and J. McAuley, "Self-Attentive Sequential Recommendation," in Proc. IEEE Int. Conf. on Data Mining (ICDM), 2018, pp. 197-206.

[2] F. Sun, J. Liu, J. Wu, C. Pei, X. Lin, W. Ou, and P. Jiang, "BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer," in Proc. ACM Int. Conf. on Information and Knowledge Management (CIKM), 2019, pp. 1441-1450.

[3] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A Simple Framework for Contrastive Learning of Visual Representations," in Proc. Int. Conf. on Machine Learning (ICML), 2020, pp. 1597-1607.

[4] A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," in Proc. Int. Conf. on Machine Learning (ICML), 2021, pp. 8748-8763.

[5] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459-9474.

[6] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A Contextual-Bandit Approach to Personalized News Article Recommendation," in Proc. Int. Conf. on World Wide Web (WWW), 2010, pp. 661-670.

[7] S. Agrawal and N. Goyal, "Analysis of Thompson Sampling for the Multi-Armed Bandit Problem," in Proc. Conf. on Learning Theory (COLT), 2012, pp. 39.1-39.26.

[8] R. J. G. B. Campello, D. Moulavi, and J. Sander, "Density-Based Clustering Based on Hierarchical Density Estimates," in Proc. Pacific-Asia Conf. on Knowledge Discovery and Data Mining (PAKDD), 2013, pp. 160-172.

[9] R. Agrawal and R. Srikant, "Fast Algorithms for Mining Association Rules," in Proc. Int. Conf. on Very Large Data Bases (VLDB), 1994, pp. 487-499.

Web

[10] Apache Software Foundation, Apache Kafka, Apache Flink and Apache Spark documentation, apache.org.

[11] Feast open-source feature store documentation, feast.dev.

[12] Qdrant vector database documentation, qdrant.tech.

[13] MLflow, Evidently AI and SHAP documentation, mlflow.org / evidentlyai.com / shap.readthedocs.io.

PROJECT DETAILS

Field	Details
Student Name(s)	D V Phani Geethika, Namita Sampath, Dhanyatha Corry
Register Number	235816128, 235816126, 235816448
Branch	Data Science
Email Address	phani.mitblr2023@learner.manipal.edu, Namita.mitblr2023@learner.manipal.edu, dhanyatha.mitblr2023@learner.manipal.edu
Phone No (M)	6361807603, 9741380516, 9743008880
Project Title	BehaviorIQ: A Real-Time, LLM-Augmented Behavioral Intelligence Platform
Faculty Guide Name	Aayushi Jain
Full Contact Address	SOCE, Manipal Institute of Technology, Bangalore - 560064(Karnataka), INDIA
Guide Email Address	jain.aayushi@manipal.edu